

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1453

**VIŠEKRITERIJSKA OPTIMIZACIJA GLAZBENIH LISTA
POMOĆU EVOLUCIJSKIH ALGORITAMA**

Tomislav Čupić

Zagreb, lipanj 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1453

**VIŠEKRITERIJSKA OPTIMIZACIJA GLAZBENIH LISTA
POMOĆU EVOLUCIJSKIH ALGORITAMA**

Tomislav Čupić

Zagreb, lipanj 2026.

DIPLOMSKI ZADATAK br. 1453

Pristupnik: **Tomislav Čupić (0036541125)**
Studij: Računarstvo
Profil: Računarska znanost
Mentor: izv. prof. dr. sc. Marko Đurasević

Zadatak: **Višekriterijska optimizacija glazbenih lista pomoću evolucijskih algoritama**

Opis zadatka:

Proučiti i opisati problem optimizacije glazbene liste kao višekriterijski optimizacijski problem. Istražiti i opisati prikladne metode za rješavanje odgovarajućeg optimizacijskog problema. Odabrati i implementirati prikladne višekriterijske evolucijske algoritme te definirati kriterijske funkcije optimizacije. Ocijeniti i usporediti učinkovitost različitih implementiranih algoritama na odgovarajućem problemu. Usporediti učinkovitost implementiranih algoritama sa slučajno generiranim listama i listama generiranim odabranom heuristikom. Analizirati i prikazati dobivene rezultate te priložiti ih radu uz dodatna objašnjenja, izvorne tekstove programa i korištenu literaturu.

Rok za predaju rada: 26. lipnja 2026.

*Zahvaljujem se mentoru izv. prof. dr. sc. Marku Đuraseviću za lijepu suradnju
tijekom akademske godine prilikom rađanja diplomskog rada.*

Sadržaj

Uvod	3
1. Spotify glazbene liste.....	4
1.1 Spotify	4
1.2 Glazbena lista kao skup podataka.....	4
2. Višekriterijska optimizacija.....	6
2.1 Optimizacija.....	6
2.2 Višekriterijski algoritam.....	6
s	
2.4 NSGA-II.....	9
2.5 NSGA-III	9
2.6 MOEA/D algoritam	10
3. Mjerenje uspješnosti algoritama	12
3.1 Mjere	12
3.2 Generacijska udaljenost	12
3.3 Mjera razmaka.....	13
3.4 Hipervolumen.....	14
4. Postavljanje i izvedba problema	15
4.1 Postavljanje problema	15
4.2 Implementacija	15
4.3 Alati i struktura	16
5. Rezultati i usporedba algoritama	18

5.1	Rezultati i međusobna usporedba algoritama	18
5.2	Usporedba s nasumičnim glazbenim listama	21
5.3	Usporedba sa ručno izabranom glazbenom listom.....	22
6.	Poboljšanja, nedostaci i daljnji rad.....	25
	Zaključak	26
	Sažetak.....	27
	Summary	28
	Literatura	29

Uvod

Razvojem digitalnih glazbenih platformi način na koji korisnici slušaju glazbu značajno se promijenio. Servisi za prijenos glazbe poput Spotifya postali su najpopularniji način organizacije i preporuke glazbenog sadržaja. Jedna od mogućnosti tih servisa je izrada glazbenih listi (eng. *playlists*) vlastoručno ili automatski. Platforma Spotify ističe se kao jedan od vodećih servisa, od kojeg je uzet skup podataka za ovaj rad, koji sadrže pjesme s njihovim audio značajkama.

Generiranje glazbene liste predstavlja problem jer je potrebno istovremeno zadovoljiti više različitih kriterija, koji mogu biti međusobno suprotstavljeni. Takav problem pripada području višekriterijske optimizacije. Za razliku od jednokriterijske optimizacije cilj nije naći jedno najbolje rješenje, već skup kompromisnih rješenja koja predstavljaju optimalnu ravnotežu između promatranih kriterija.

Jedan od najčešćih pristupa rješavanju višekriterijskih optimizacijskih problema je primjena evolucijskih algoritama. Evolucijski algoritmi inspirirani su prirodnom evolucijom te pretražuju veliki broj mogućih rješenja omogućujući približavanje optimalnoj Pareto fronti. U ovom radu korištena su tri poznata algoritma: NSGA-II, NSGA-III te MOEA/D algoritam, koji primjenjuju različite strategije konvergencije i raznolikosti rješenja.

Cilj ovog rada je implementirati navedene algoritme u Pythonu, primijeniti ih na problem generiranja glazbenih listi te međusobno ih usporediti. Rješenja se međusobno uspoređuju mjerama konvergencije i raznolikosti kako bi se procijenila uspješnost algoritma.

Rad je organiziran da se prvo predstavi teorijska osnova višekriterijske optimizacije i samih algoritama. Nakon toga je opisana implementacija algoritama te mjere korištene za ocjenjivanje rješenja. U završnom djelu su opisani rezultati, usporedba algoritama te analiza njihove učinkovitosti.

1. Spotify glazbene liste

1.1 Spotify

Spotify je popularna švedska usluga za prenošenje glazbe i auditivnih sadržaja koju su osnovali Daniel Ek i Martin Lorentzon 2006. godine. Jedna je od najprodavanijih servisa za prijenos glazbe sa 761 milijunom aktivnih korisnika u 2026. godini od kojih je 293 milijuna pretplaćenih na „premium“ verziju aplikacije. Sadrži preko 100 milijuna pjesama te 7 milijuna „podcasta“. Glazba se može pretraživati po žanru, izvođaču, albumu, popisu pjesama. Izgled početka jedne glazbene liste u Spotifyu može se vidjeti na (Sl. 1.1), na vrhu slike prikazane su labele liste: naziv, album, datum dodavanja i vrijeme trajanja svake pjesme [1].

#	Naziv	Album	Datum dodavanja	
1	 Wherever I Go Spot • OneRepublic	Oh My My (Deluxe)	18. stu 2025.	2:50
2	 Toxic BoyWithUke	Toxic	18. stu 2025.	2:48
3	 Bad Habits Spot • Ed Sheeran	=	18. stu 2025.	3:51
4	 Love Me Now Spot • John Legend	DARKNESS AND LIGHT	18. stu 2025.	3:30
5	 Come & Go (with Marshmello) Spot • Juice WRLD, Marshmello	Legends Never Die	18. stu 2025.	3:25
6	 Centuries Spot • Fall Out Boy	American Beauty/American...	18. stu 2025.	3:48
7	 Better Now Spot • Post Malone	beerbongs & bentleys	18. stu 2025.	3:51
8	 Tarantella Gabry Ponte, KEL	Tarantella	5. pro 2025.	2:26
9	 We Don't Talk Anymore (feat. Selena Go... Spot • Charlie Puth, Selena Gomez	Nine Track Mind (Deluxe Ed...	28. ožu 2026.	3:38

Sl. 1.1 Spotify glazbena lista

1.2 Glazbena lista kao skup podataka

Glazbena lista je lista audio i video datoteka koja se može izvoditi na „medijskom playeru“. Može biti sortirana u niz ili nasumična. U najužem smislu je lista skladbi koja se može izvoditi jednom ili više puta u petlji [2]. Skup podataka od 30 tisuća pjesama nađen je na „Kaggle“ stranici [3]. U njemu je tablica koja sadrži sljedeće varijable: oznaka i ime pjesme, ime izvođača, popularnost, ime i oznaka albuma, žanr, datum

objave, ime i oznaka glazbene liste, žanr liste, te atributi same pjesme: plesnost, energija, ključ, glasnoća, modalitet, količina govora, akustičnost, instrumentalnost, živost, pozitivnost, tempo te duljina same pjesme. Za rad nisu bili potrebni svi navedeni atributi pa su u (Tablica 1.1) izdvojeni oni bitni.

Tablica 1.1 izdvojeni atributi

Atribut	Tip	Opis
Ime pjesme	Tekst	Puni naslov pjesme
Oznaka pjesme	Tekst	Jedinstvena oznaka pjesme
Ime izvođača	Tekst	Ime i prezime izvođača
Popularnost pjesme	Broj <0,100>	Koliko je pjesma popularna
Energija	Broj <0, 1>	Mjera intenziteta i aktivnosti pjesme
Glasnoća	Broj	Glasnoća pjesme u decibelima
Pozitivnost	Broj <0, 1>	Mjera koliko pjesma zvuči sretno i euforično
Tempo	Broj	Brzina, odnosno korak pjesme mjereno u otkucajima po minuti

2. Višekriterijska optimizacija

2.1 Optimizacija

Optimizacija je proces izmjene nekog rješenja gdje se ono modificira u svrhe ubrzanja rada ili manjeg iskorištavanja resursa [4]. To je problem u kojem želimo pronaći dovoljno dobro rješenje. Svako rješenje možemo ocijeniti funkcijom dobrote ili kvalitete. Cilj optimizacije je pronaći što bolje rješenje u što manje koraka.

Optimizacijski problemi se dijele:

- S obzirom na domenu:
 - Kontinuirani
 - Diskretni
- Prema ograničenjima:
 - Bez ograničenja
 - S ograničenjima
- S obzirom na broj kriterija:
 - Jednokriterijski
 - Višekriterijski

Ukratko, optimizacijski problem sastoji se od funkcije koja se maksimizira ili minimizira, te ograničenjima koja ograničavaju prostor pretrage odnosno vrijednosti koje varijable mogu poprimiti [5].

Problem koji ovaj rad proučava je višekriterijski diskretni problem bez ograničenja.

2.2 Višekriterijski algoritam

Višekriterijski algoritmi su oni koji optimiziraju u odnosu na različite kriterije koji su najčešće konfliktne. Može se zapisati kao:

Minimiziraj ili maksimiziraj: $f_m(\vec{x})$, $m = 1, 2, \dots, M$

Uz ograničenja: $g_j(\vec{x}) \geq 0$, $j = 1, 2, \dots, J$

$h_k(\vec{x}) \geq 0$, $k = 1, 2, \dots, K$

$$x_i^{(L)} \leq x_i \leq x_j^{(L)}$$

Gdje M predstavlja broj kriterija koje je potrebno optimizirati.

U višekriterijskim problemima često imamo više rješenja koja su jednako dobra. To je zato što po jednom kriteriju jedno rješenje može biti bolje od drugog rješenja, a po drugom kriteriju lošije od tog istog rješenja. Dominacija je odnos rješenja gdje neko rješenje dominira nad drugim po svim kriterijima, odnosno x_1 dominira nad x_2 ako vrijedi:

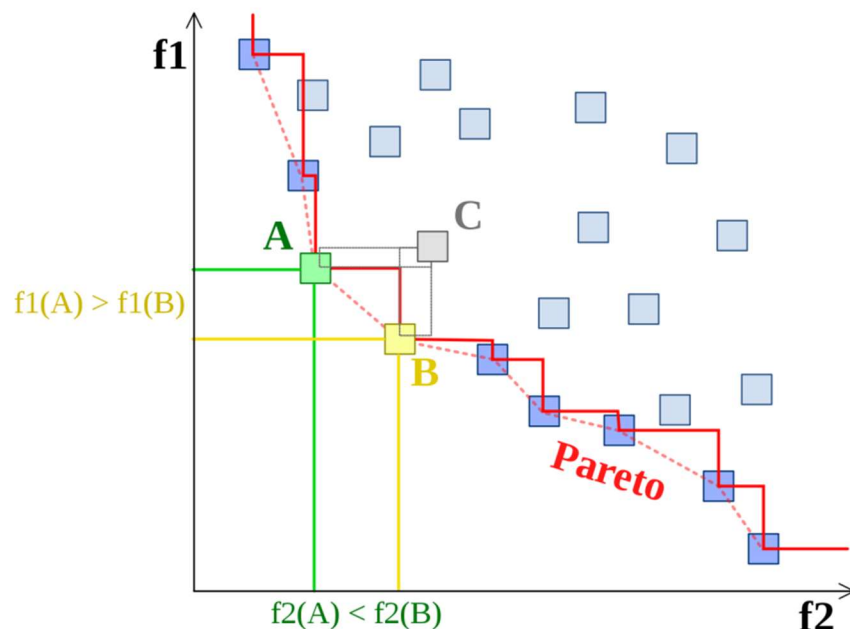
$$f_i(x_1) \leq f_i(x_2), \forall i \in \{1, 2, \dots, k\}$$

te

$$f_j(x_1) < f_j(x_2), \text{ za barem jedan } j \in \{1, 2, \dots, k\}$$

Cilj višekriterijske optimizacije je pronaći skup rješenja nad kojim niti jedno drugo rješenje ne dominira. Taj skup naziva se nedominirani skup, a Pareto optimalni skup je skup svih nedominiranih rješenja koja zadovoljavaju sva ograničenja problema. Pareto fronta čiji primjer je prikazan slikom dolje (Sl. 2.1), skup je vrijednosti rješenja iz Pareto optimalnog skupa u prostoru kriterija.

Pareto fronta



Sl. 2.1 Pareto fronta

Cilj je pronaći što bolju aproksimaciju prave Pareto fronte. Rješenja su sortirana u fronte te osim najbolje konvergencije tražena je i dobra raznolikost rješenja. Jedan od

načina sortiranja je nedominirano sortiranje koje se koristi u NSGA algoritmima. Sortiranje funkcionira tako da se svakom rješenju dodijeli rang i skup rješenja nad kojima ono dominira. U prvu frontu su stavljena rješenja koja nisu dominirana od bilo kojeg drugog rješenja u populaciji te im se dodijeli rang nula te se ide po njegovom skupu i tim rješenjima se smanji rang za 1. Tako dobijemo sljedeći skup rješenja koji ima rang nula. Taj postupak se ponavlja dok ne dobijemo sva rješenja u frontama.

2.3 Evolucijski algoritam

U računalnoj inteligenciji, evolucijski algoritam je generički metaheuristički optimizacijski algoritam zasnovan na populacijama. Koristi mehanizme nadahnute biološkom evolucijom, poput mutacije, križanja, selekcije i reprodukcije [6]. Kandidati za optimalna rješenja su jedinke populacije koje pokušavaju prilagoditi se da prežive, a njihova kvaliteta određena je funkcijom dobrote (engl. *fitness*). Operatori evolucijskih algoritama su:

- Selekcija (engl. *selection*) – biranje najboljih jedinki za razmnožavanje
- Križanje (engl. *crossover*) – križanje gdje se uzima jedan dio rješenja prvog roditelja i jedan dio drugog roditelja te se dobije dijete s genima oba roditelja
- Mutacija (engl. *mutation*) – nasumično mijenjanje jednog dijela (gena) rješenja s drugim radi poticanja eksploracije rješenja

Početna populacija, odnosno prva generacija algoritma većinom se generira nasumično. Nakon toga ponavljamo genetske operatore nad populacijom do završetka optimizacije:

- Procijenimo funkciju dobrote za sva rješenja te odabiremo najbolja
- Razmnožavamo jedinke koristeći operator križanja te neke mutiramo nakon križanja
- Zamijenimo rješenja s najmanjom funkcijom dobrote s novim jedinkama

2.4 NSGA-II

Jedan od najpopularnijih algoritama višekriterijske optimizacije je NSGA-II algoritam (engl. *Non-dominated sorting genetic algorithm II*) koji predstavlja poboljšanu verziju izvornog NSGA algoritma. Slijed algoritma prikazan je pseudokodom 2.1.

```
Inicijaliziraj populaciju P
Dok(kriterij zaustavljanja nije zadovoljen)
    Q = Nova populacija nastala provođenjem operatora nad P
    R = Q U P
    F = Nedominirano sortiraj R u fronte
    C = Nova populacija
    i = 0
    Dok (|C| + |Fi| < |P|)
        Izračunaj crowding distance za Fi
        C = C U Fi
        i++
    Kraj Dok
    Izračunaj crowding distance za Fi
    Sortiraj Fi po crowding distanceu
    C = C U Fi[1:N - |C|]
    P = C
Kraj dok
```

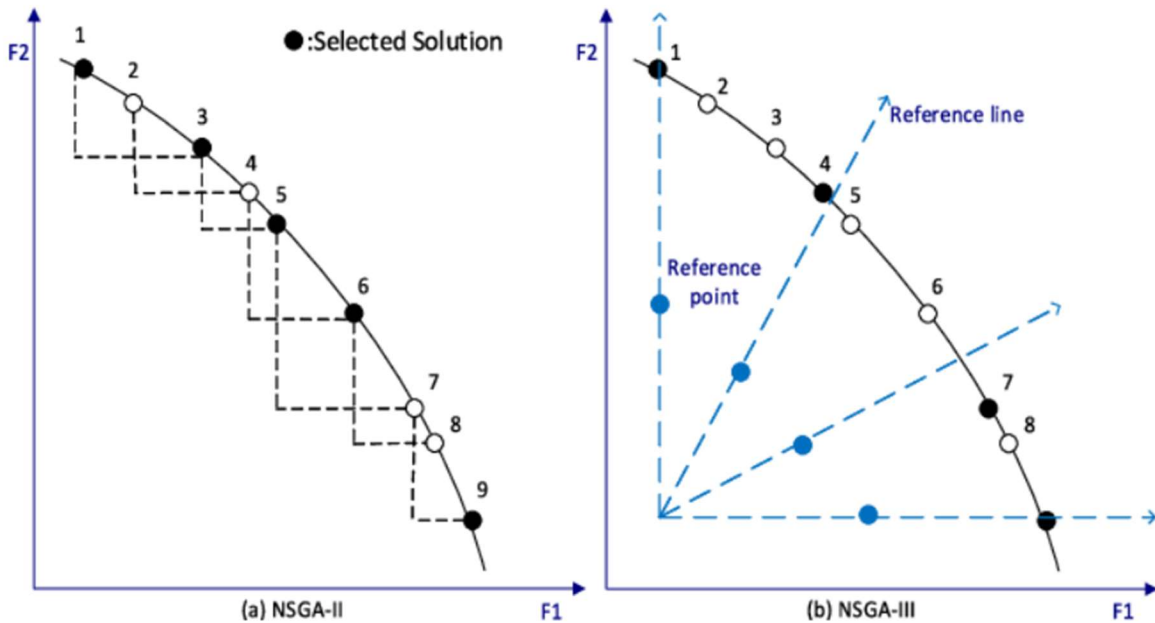
2.1 Pseudokod NSGA-II algoritma

Za razliku od običnog NSGA algoritma, NSGA-II uvodi elitizam, odnosno najbolja rješenja će uvijek biti prenesena u sljedeću generaciju jer im je rang najmanji. Raznolikost rješenja održava se primjenom mjere „crowding distance“, koja procjenjuje gustoću rasporeda rješenja u okolini pojedine jedinke. Kod NSGA-II „crowding distance“ se gleda pri zadnjoj fronti koja ulazi u sljedeću generaciju. Cijela fronta ne stane u novu populaciju, pa tada prednost u toj fronti imaju jedinke sa većom udaljenošću.

2.5 NSGA-III

Sljedeći algoritam korišten u radu je NSGA-III (engl. *Non-dominated sorting genetic algorithm III*) algoritam, koji je nadogradnja NSGA-II algoritma te se od njega razlikuje po računanju „crowding distancea“. Ovdje se računa koristeći Das-Dennis metodu

nalaženja referentnih pravaca. Das-Dennis metoda generira referentni skup točaka koje leže na „simplexu“ te uniformno pokrivaju prostor. Svaka jedinka se pridruži najbližem referentnom pravcu, zatim se gleda koliko rješenja svaki od referentnih pravaca ima pridruženo, te prednost dobiju rješenja blizu najmanje popunjenih pravaca [7]. Razlike između NSGA-II i NSGA-III načina za računanje „crowding distancea“ mogu se vidjeti na (Sl. 2.2).



Sl. 2.2 „Crowding distance“ NSGA-II i NSGA-III algoritama

2.6 MOEA/D algoritam

Treći algoritam korišten u radu je MOEA/D (engl. *A Multiobjective Evolutionary Algorithm Based on Decomposition*). Za razliku od NSGA algoritama MOEA/D algoritam ne radi s Pareto dominacijom i referentnim smjerovima, već radi razlaganje (dekompoziciju) višekriterijskih problema na više skalarnih optimizacijskih potproblema [8]. Slijed algoritma prikazan je pseudokodom 2.2.

```

Inicijaliziraj populaciju P
Generiraj N težinskih vektora  $\lambda$ 
Izgradi strukturu susjedstva B za svaki  $\lambda$ 
Inicijaliziraj referentnu točku  $z^*$ 
Dok(kriterij zaustavljanja nije zadovoljen):
    Za svaku jedinku iz P:
        Primjeni genetske operatore i generiraj dijete
        Ažuriraj  $z^*$ 
        Za svakog susjeda j iz B(i):
            Izračunaj Tchebycheffovu vrijednost djeteta i
            trenutnog rješenja
            Ako (vrijednost djeteta  $\leq$  trenutnog
            rješenja):
                Vrijednost trenutnog rješenja =
                vrijednost djeteta
            Kraj Ako
        Kraj Za
    Kraj Dok
Vrati populaciju P

```

2.2 pseudokod MOEA/D algoritma

Pri dekompoziciji problema kvaliteta rješenja procjenjuje se pomoću Tchebycheffove skalarizacijske funkcije i referentne točke z^* , dok se nova rješenja uspoređuju samo s rješenjima u svom susjedstvu. Takav pristup omogućuje učinkovito pretraživanje prostora rješenja i dobru raspodjelu aproksimacija duž Pareto fronte [9].

3. Mjerenje uspješnosti algoritama

3.1 Mjere

Efikasnost algoritama je međusobno uspoređena koristeći tri različite mjere, generacijsku udaljenost, raspršenost (engl. *spread*), te hipervolumen. U nastavku će biti objašnjena svaka od tih mjera. Generacijska udaljenost korištena je da se može razaznati konvergencija algoritama, raspršenost raznolikost algoritama, te hipervolumen kao mjera koja uključuje i konvergenciju i raznolikost algoritama.

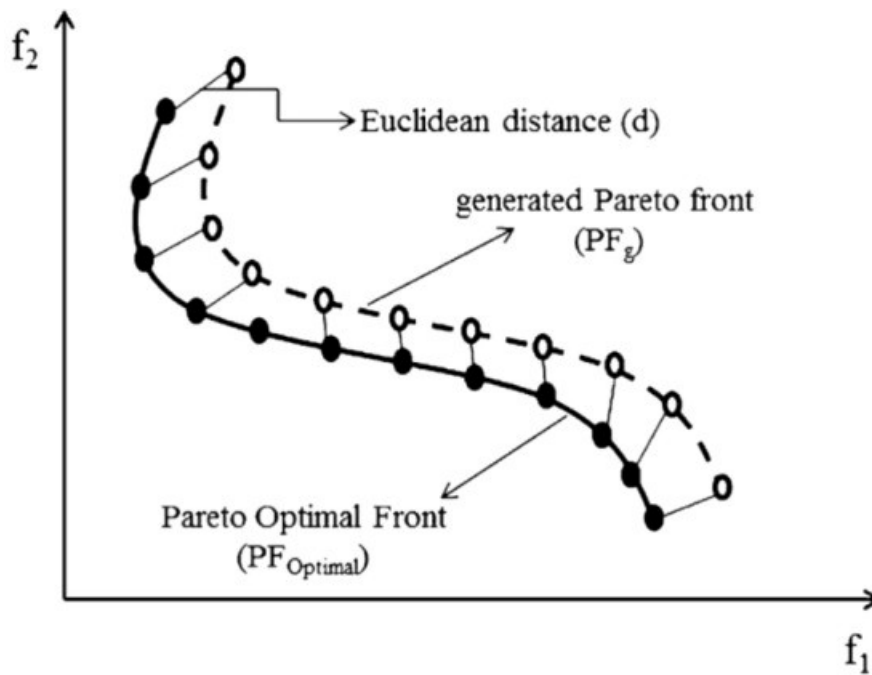
3.2 Generacijska udaljenost

Prva mjera koja je gledana nad algoritmima je generacijska udaljenost (engl. *generational distance*), jedna od najčešće korištenih mjera za procjenu konvergencije algoritma prema stvarnoj Pareto fronti. Opisali su ju Van Veldhuizen i Lamont te referira se kao udaljenost između generirane Pareto fronte i stvarne optimalne Pareto fronte.

Formula po kojoj se računa glasi: $GD = (\frac{1}{N} \sum_{i=1}^N d_i^p)^{1/p}$ gdje su:

- N – broj rješenja u dobivenoj Pareto aproksimaciji
- d_i – euklidska udaljenost između i -tog rješenja i najbliže točke referentne Pareto fronte
- p – parametar norme, najčešće se koristi broj dva

Primjer računanja prikazan je na (Sl. 3.1).



Sl. 3.1 Generacijska udaljenost

3.3 Mjera razmaka

Sljedeća mjera koja je korištena je mjera razmaka (engl. *spacing*). Ona je mjera performansi algoritama koja procjenjuje ravnomjernost raspodjele nedominiranih rješenja na Pareto fronti. Njena svrha je utvrditi koliko su rješenja međusobno jednoliko raspoređena, odnosno postoji li grupiranje rješenja u pojedinim dijelovima fronte ili su ravnomjerno raspodijeljena duž cijele fronte. Matematički se opisuje kao:

$$S = \sqrt{\frac{1}{NPF - 1} \sum_{i=1}^{NPF} (d_i - \vec{d})^2}$$

gdje je:

- NPF – broj rješenja u Pareto fronti
- d_i – najmanja udaljenost između i-tog rješenja i njemu najbližeg susjednog rješenja
- $\vec{d}$ – srednja udaljenost svih udaljenosti d_i

Vrijednost metrike predstavlja standardnu devijaciju udaljenosti susjednih točaka. Manje vrijednosti mjere razmaka ukazuju na bolju raznolikost i umjereniju raspodjelu rješenja, dok veće vrijednosti upućuju na neravnomjernu raspodjelu, odnosno postoje

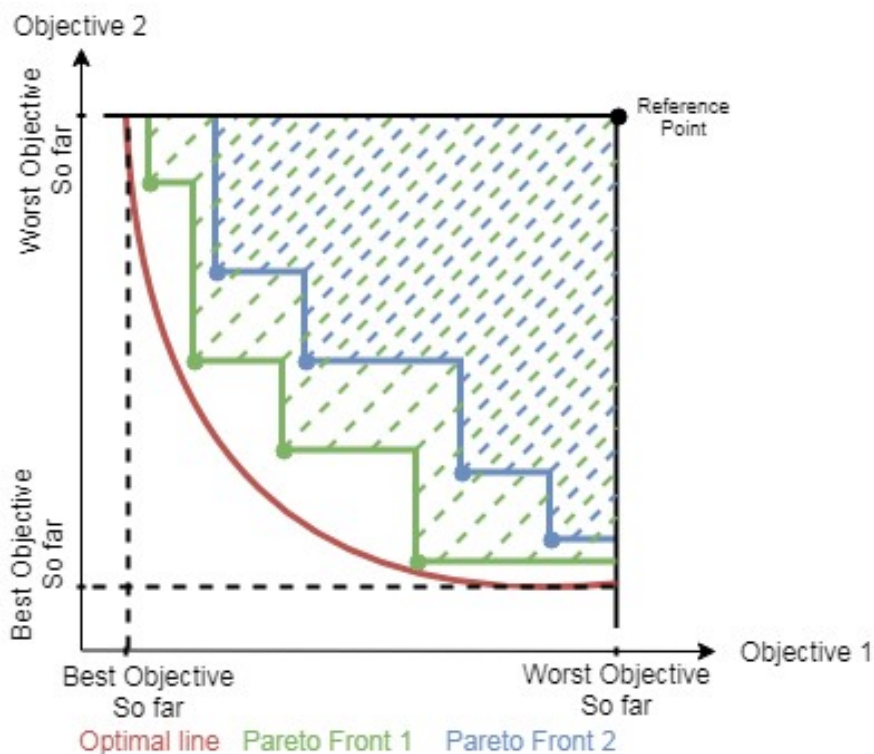
grupirana područja i područja u kojima nedostaje rješenja. Zbog toga se koristi kao mjera raznolikosti rješenja ali ne i kao mjera konvergencije. Algoritam može imati dobru raspodjelu rješenja, ali da su vrlo udaljena od optimalne Pareto fronte [10].

3.4 Hipervolumen

Osim mjera za konvergenciju i raznolikost za svaki algoritam izračunat je hipervolumen. Hipervolumen jedna je od najčešćih mjera koja predstavlja volumen prostora ciljeva dominiran skupom Pareto rješenja i referentnom točkom. Veći hipervolumen znači da skup rješenja istodobno:

- Bolje aproksimira Pareto frontu
- Pokriva veći dio prostora kompromisa između ciljeva

Zbog toga što istodobno vrednuje konvergenciju i raznolikost rješenja, hipervolumen jedan je od najboljih pokazatelja kvalitete u višekriterijskoj optimizaciji. Primjer izgleda hipervolumena prikazan je na (Sl. 3.2). Iz slike se primjećuju hipervolumeni dviju Pareto fronta, gdje je zelenom bojom označena fronta s većim hipervolumenom te se može uočiti da je bliža crti koja predstavlja rješenja s optimalne Pareto fronte [11].



Sl. 3.2 Hipervolumen

4. Postavljanje i izvedba problema

4.1 Postavljanje problema

Jedinka u genetskim algoritmima predstavlja jednu glazbenu listu, odnosno listu indeksa pjesama odabranih iz skupa podataka. Veličina jedinke zadaje se prije pokretanja algoritma, u radu je postavljena na 100 pjesama. Problem opisan u radu postavljen je tako da su od navedenih atributa iz zadanog skupa podataka izabrana tri korištena u optimizaciji:

- **Popularnost pjesme:** prikazana je atributom popularnost koji je među osnovnim atributima, te se ona maksimizirala. Mogućnost je nju i minimizirati ako je željeni rezultat otkriti novu glazbu (engl. *discovery playlist*). U radu je maksimizirana jer je željeni rezultat svakodnevna glazbena lista za slušanje, a ne za otkrivanje nove muzike. Vrlo poznate pjesme u istoj listi s potpuno nepoznatima ne bi rezultirale nijednom navedenom glazbenom listom, već bi nastala nebalansirana lista.
- **Različitost izvođača:** računa se pomoću atributa izvođač pjesme, gdje je u rezultatu, odnosno dobivenoj glazbenoj listi sumiran broj različitih izvođača. Različitost je u radu maksimizirana jer bi pri minimiziranju rezultat bio glazbene liste koje su većinom samo jedan izvođač. Pjesme istog izvođača su većinom vrlo slične pa bi u prvoj Pareto fronti bile samo takve liste.
- **Koherentnost:** vektor atributa koji sadrži energiju, glasnoću, pozitivnost i tempo pjesme. Vektoru je minimizirana varijanca značajki, tako da pjesme u glazbenoj listi međusobno sličnije zvuče. Pjesme sa manjom varijancom koherentnosti predstavljaju glazbeno ujednačeniju listu.

4.2 Implementacija

Parametri su izabrani koristeći pretraživanje mreže parametara (engl. *grid search*). Pretraživanje mreže parametara je metoda optimizacije hiperparametara koja definira skup mogućih vrijednosti za svaki parametar te generira mrežu svih međusobnih kombinacija vrijednosti. Za svaku kombinaciju zatim se evaluira rješenje, te najbolje

rješenje prema funkciji dobrote je ono koje je uzeto za daljnja pokretanja algoritama. Iz skupa podataka od 30 000 pjesama, za genetske algoritme početna populacija sadrži 100 jedinki, te je svaki algoritam pokrenut s 200 generacija. Također je pretraživanje obrađeno na različitim vjerojatnostima mutacije te najbolja dobivena vrijednost iznosi 15%. Za operator križanja korištena je jedna točka križanja do koje su se uzimale pjesme iz jednog roditelja, a nakon koje su se uzimale pjesme iz drugog roditelja. Mutacija se obavlja tako da se pri zadanoj vjerojatnosti jedna pjesma iz glazbene liste zamijeni nasumično izabranom pjesmom iz zadanog skupa pjesama. Obavlja se provjera nalazi li se nova pjesma već u rješenju, te se u tom slučaju ponovno izabire pjesma. Algoritam MOEA/D koristi veličinu susjedstva od 10.

Značajke koje su implementirane kao vektor koherentnosti pjesme, što je jedan od kriterija optimizacije, normalizirane su tako da su sve na intervalu $[-1, 1]$, što je omogućilo jednostavnije računanje i grafički prikaz rezultata. Značajke su razmatrane tako da je broj izvođača suma različitih izvođača na glazbenoj listi podijeljena sa duljinom liste, popularnost je računata kao srednja vrijednost popularnosti svih pjesama normalizirana na zadani interval, a koherentnost je izračunata kao srednja vrijednost varijance vektora pjesama.

Izračunate su metrike hipervolumena, generacijske udaljenosti i mjere razmaka. Za izračun hipervolumena korištena je referentna točka postavljena na vrijednost 1,1 u svakoj dimenziji prostora ciljnih funkcija, odnosno $[1,1; 1,1; 1,1]$ pri čemu su sve funkcije prethodno normalizirane. Za izračun generacijske udaljenosti korištena je aproksimacija prave Pareto fronte konstruirana kao skup svih nedominiranih rješenja iz unije svih fronti iz trideset pokretanja svih triju algoritama. Na taj način referentni skup obuhvaća najbolja poznata rješenja bez pristranosti prema nekom algoritmu.

4.3 Alati i struktura

Navedeni algoritmi samostalno su implementirani u programskom jeziku Python, koristeći „numpy“, „pandas“, „math“ i „random“ module za zapisivanje rezultata i kalkulacije. Testiranja algoritama su rađena koristeći „Jupyter notebook“, a algoritmi su napisani u zasebnim Python datotekama.

Grafovi su izrađeni pomoću „matplotlib“ biblioteke Pythona, koja na jednostavan način prikazuje različite tipove grafova. U radu su od „matplotlib“ funkcija korištene „boxplot“ i „bar“ funkcije za kutijaste, odnosno stupčaste grafove.

Napravljena je klasa *PlaylistObjectives* koja implementira način evaluiranja svake značajke. Zatim su napravljene zasebne klase za svaki algoritam, *NSGA-II*, *NSGA-III* i *MOEA/D*, koje implementiraju navedene algoritme, te klasa *Individual* za svaki algoritam. Klasa *Individual* predstavlja jednu glazbenu listu, odnosno jedinku algoritma. Ona sadrži funkcije mutacije i evaluacije, tako da se jedinka može ocijeniti pomoću funkcija iz klase *PlaylistObjectives*.

Također je napravljena klasa *Hypervolume*, te statičke funkcije *Generational_distance()* i *Spacing()*, koje su računaju navedene metrike za rezultate.

Za usporedbu rezultata, svaki algoritam pokrenut je trideset puta te su na temelju dobivenih srednjih vrijednosti i njihovih standardnih devijacija dobiveni prikladni rezultati i prikazani grafovi.

5. Rezultati i usporedba algoritama

5.1 Rezultati i međusobna usporedba algoritama

Svaki algoritam pokrenut je trideset puta, te su uzete srednje vrijednosti njihovih metrika tijekom tih pokretanja. Za razliku od NSGA algoritama, MOEA/D algoritam ne dijeli rješenja u Pareto fronte, ali iz konačnog skupa rješenja može se pogledati koja rješenja su nedominirana, pa njih sumirati kao veličinu Pareto fronte. Za računanje mjere raznolikosti potrebno je koristiti isti broj rješenja, pa su za nju skalirana na najmanji broj rješenja optimalne fronte od sva tri algoritma.

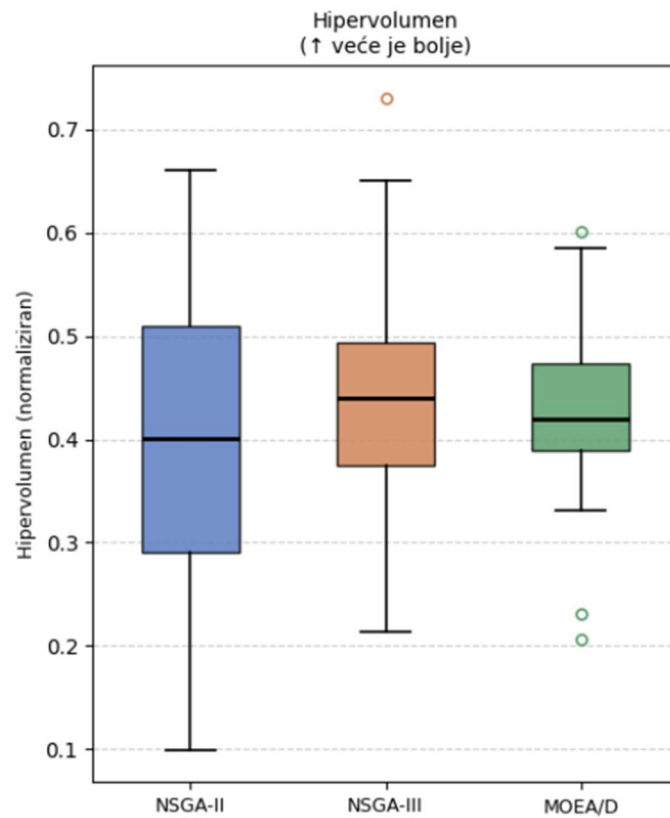
Dobiveni rezultati prikazani su (Tablica 5.1), mjere su opisane kao srednja vrijednost $\pm$ standardna devijacija.

Tablica 5.1 usporedba algoritama

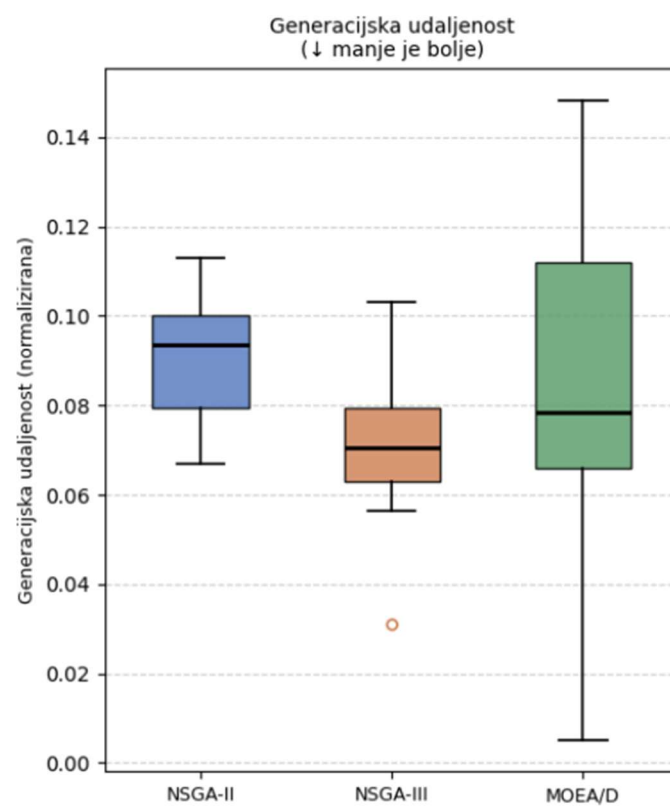
Algoritam	Hipervolumen	Generacijska udaljenost	Mjera razmaka
NSGA-II	0.3947 ± 0.1521	0.09063 ± 0.01303	0.04724 ± 0.01655
NSGA-III	0.4429 ± 0.1165	0.07132 ± 0.01332	0.04729 ± 0.01312
MOEA/D	0.4265 ± 0.0876	0.08442 ± 0.03462	0.02795 ± 0.01975

Iz priložene (Tablica 5.1) može se vidjeti da je najbolja raznolikost kod algoritma MOEA/D, najbolja konvergencija kod algoritma NSGA-III, a najveći hipervolumen također kod algoritma NSGA-III.

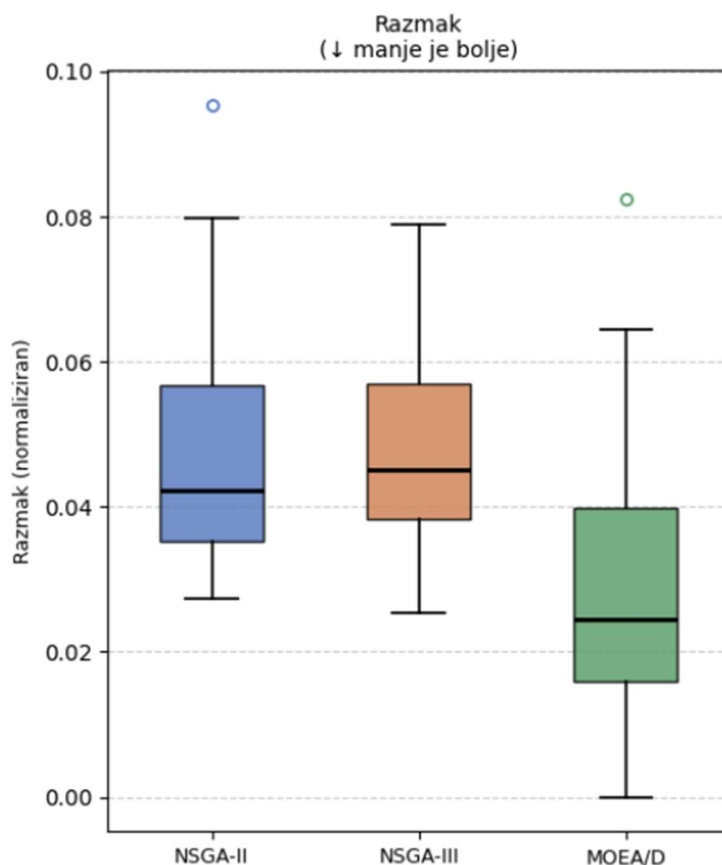
Prikaz rješenja algoritama grafički je napravljen koristeći Matplotlib biblioteku. Napravljeni su kutijasti grafovi (engl. *box*) za hipervolumen (Sl. 5.1), generacijsku udaljenost (Sl. 5.2) te mjeru razmaka (Sl. 5.3). Iz njih se mogu lako iščitati kvartili rješenja, maksimumi, minimumi i odstupajuće vrijednosti koje su na grafu označene nepopunjenim krugovima.



Sl. 5.1 Kutijasti dijagram hipervolumena



Sl. 5.2 Kutijasti dijagram generacijske udaljenosti



Sl. 5.3 Kutijasti dijagram mjere razmaka

Iz priloženih grafova može se uočiti da je po hipervolumenu NSGA-III algoritam najbolji, jer ima malu disperziju između različitih pokretanja a visoku vrijednost hipervolumena. Mjera razmaka najbolja je kod MOEA/D algoritma, no generacijska udaljenost, odnosno konvergencija je vrlo loša. Također je najveća disperzija rješenja kod algoritma MOEA/D što ukazuje da nije vrlo pouzdano rješenje. Mjera razmaka između NSGA-II i NSGA-III algoritma je vrlo slična, no konvergencija računata generacijskom udaljenošću je mnogo bolja kod NSGA-III. Zaključno, po navedenim rezultatima NSGA-III je najbolji algoritam za zadani problem generiranja glazbenih listi. NSGA-II i MOEA/D algoritmi također nisu loši na zadanom zadatku, te su i oni uspješni u generiraju glazbenih lista.

Rezultati najuspješnijeg algoritma NSGA-III prikazani su (Tablica 5.2).

Tablica 5.2 najbolji rezultat NSGA-III algoritma

Koherentnost	0.0096
Popularnost	62.54
Diverzifikacija izvođača	1

Iz (Tablica 5.2) se uočava da je NSGA-III veoma uspješno rješenje našeg problema. Različitost izvođača jednaka jedan predstavlja glazbenu listu gdje se niti jedan izvođač nije ponovio, izabrane su popularne pjesme te je koherentnost iznimno malena.

5.2 Usporedba s nasumičnim glazbenim listama

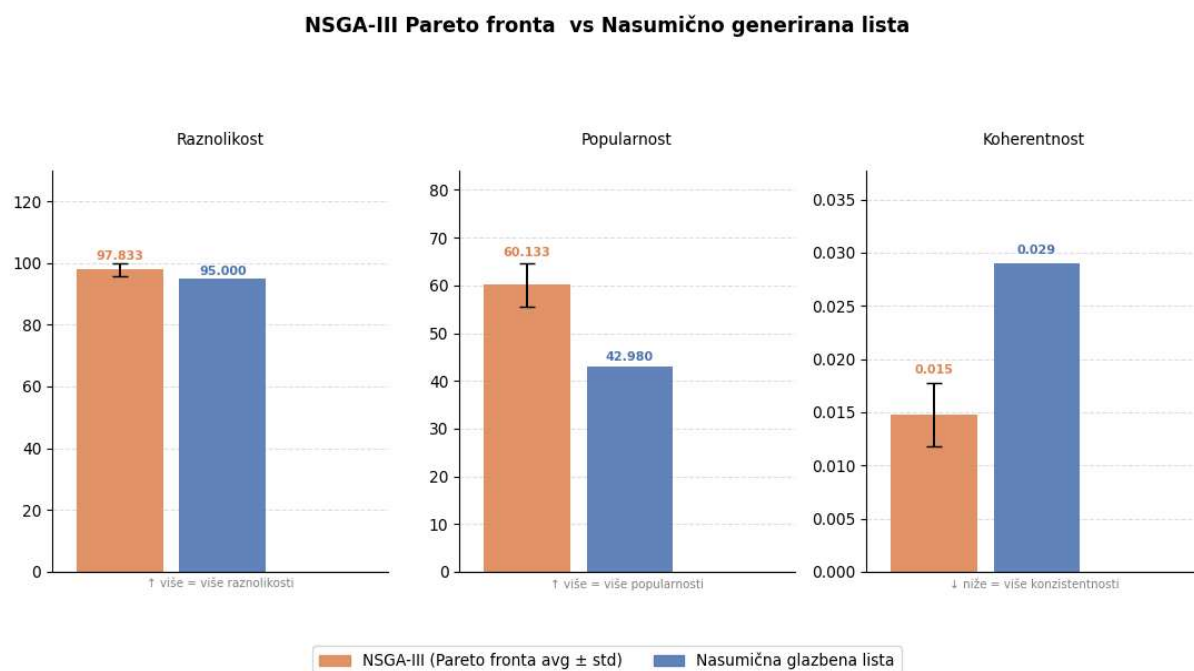
Iz skupa podataka nasumično su generirane glazbene liste od 100 pjesama, te je napravljena Pareto fronta od najboljih nasumično generiranih rješenja za bolju usporedbu. Glazbena lista uspoređena je s NSGA-III algoritmom budući da je on ostvario najbolje rezultate od tri implementirana algoritma. Mjerene su i uspoređene tri značajke po kojima optimiziramo. Značajke po uspoređenim algoritmima prikazani su (Tablica 5.3).

Tablica 5.3 Usporedba nasumično generiranih glazbenih lista i NSGA-III algoritma

Značajke	Algoritam NSGA-III	Nasumično generirana glazbena lista
Broj Različitih izvođača	99.0 ± 0.1114	95
Popularnost	58.4832 ± 4.6557	42.98
Koherentnost	0.0138 ± 0.0019	0.029

Iz (Tablica 5.3) može se vidjeti da algoritam NSGA-III koji je implementiran u radu uspješniji od nasumično generirane glazbene liste po svim značajkama. Broj različitih izvođača je očekivano jako visok u oba slučaja, zato što nasumično generirana lista ima 100 pjesama, pa su velike šanse da se od 30000 pjesama izabere mnogo različitih izvođača. Popularnost NSGA-III algoritma je mnogo veća od nasumične liste, te najveća razlika je u koherentnosti, odnosno optimizirani vektor audio značajki. Gotovo dvostruko veća koherentnost je u nasumično generiranoj glazbenoj listi. Rezultati ukazuju da je implementirani algoritam mnogo bolji od nasumično generiranih lista te da uspješno optimizira zadani problem.

Usporedba algoritma s nasumično generiranom glazbenom listom prikazana je grafički koristeći stupčasti dijagram prikazan na (Sl. 5.4).



Sl. 5.4 usporedba NSGA-III sa nasumično generiranom listom

5.3 Usporedba sa ručno izabranom glazbenom listom

Od navedenih 30 tisuća pjesama izabrano je 65 pjesama te je vlastoručno napravljena glazbena lista. Lista je napravljena tako da je strastveni slušatelj žanra „rap“ ručno izabrao pjesme. Glazbena lista zato sadrži samo pjesme koje su tog žanra, tako da se to može smatrati simuliranjem heuristike. Također je osoba upoznata s metrikom

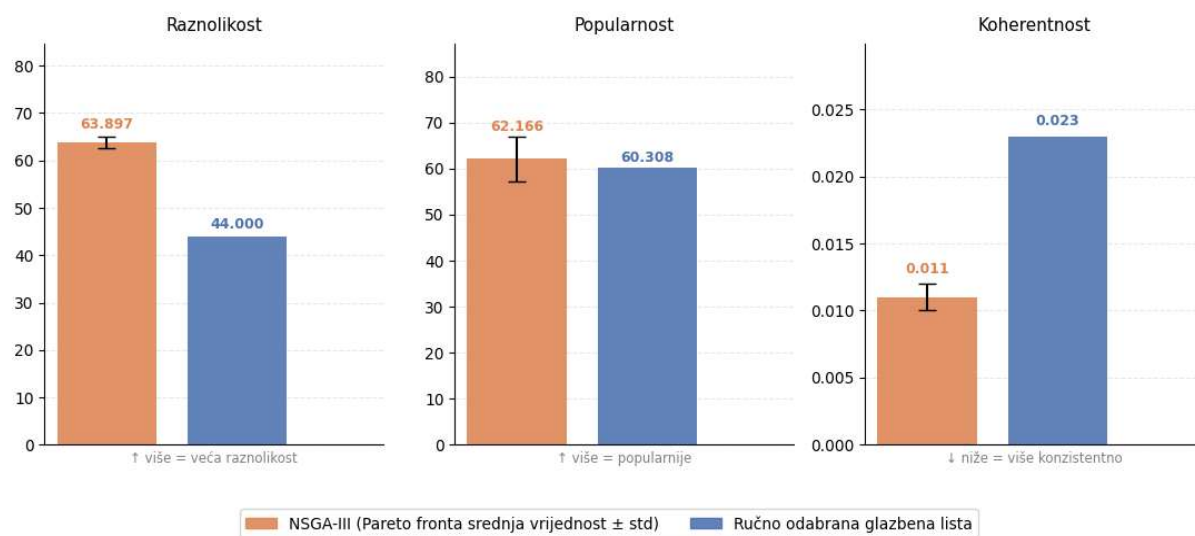
popularnošću pa nije birala nepoznate pjesme. Napravljena je usporedba ručno napravljene glazbene liste i prosječnog dobivenog rješenja NSGA-III algoritma te dobiveni rezultati prikazani su (Tablica 5.4).

Tablica 5.4 usporedba lista s heuristikom i NSGA-III

Značajke	Algoritam NSGA-III	Ručno napravljena glazbena lista
Broj Različitih izvođača	63.8966 ± 1.2414	44
Popularnost	62.1655 ± 4.8772	60.31
Koherentnost	0.0113 ± 0.0014	0.0232

Iz (Tablica 5.4) može se primijetiti da je algoritam NSGA-III implementiran u radu uspješniji od ručno napravljene glazbene liste. Maksimizira se broj različitih izvođača te su od 65 pjesama gotovo svi različiti izvođači. Koherentnost koja je minimizirana je dvostruko bolja od ručno napravljene glazbene liste. Jedino popularnost je u oba slučaja vrlo dobra jer su u ručno biranoj glazbenoj listi birane popularnije pjesme iz žanra. Većina rješenja iz NSGA-III algoritma su dominirala nad ručno izabranim rješenjem, odnosno bila su bolja po svim značajkama. Dvadeset četiri od dvadeset devet rješenja su dominirala, dok 5 njih nisu, što je veliki uspjeh za NSGA-III algoritam. Ne možemo sa sigurnošću reći da bi NSGA-III algoritam bio uspješniji od ručno izabranih glazbenih listi po ovim značajkama, specifično da dominira nad njima, no ovo je dobar pokazatelj za daljnji rad. Usporedba značajki navedenih rezultata prikazana je i grafički (Sl. 5.5), gdje je narančasti, odnosno lijevi stupac NSGA-III algoritam, a desni stupac je ručno generirana glazbena lista.

NSGA-III i ručno odabrana lista — karakteristike



Sl. 5.5 Usporedba značajki NSGA-III algoritma i ručno izabrane glazbene liste

6. Poboljšanja, nedostaci i daljnji rad

Iako su implementirani algoritmi NSGA-II, NSGA-III te MOEA/D među najpoznatijim i najčešće korištenim algoritmima u višekriterijskoj optimizaciji, ne može se tvrditi da su optimalan izbor za generiranje glazbenih lista. Prema „No free lunch“ teoremu ne postoji algoritam koji je za sve probleme najbolji. Poboljšanje algoritma za neku kategoriju problema bit će popraćeno lošijim performansama na drugoj kategoriji. Stoga, iako su algoritmi prikazivali zadovoljavajuće rezultate moguće je stvoriti ili prilagoditi neki algoritam koji bi bio učinkovitiji za ovu domenu. Poboljšanje koje bi se moglo vidjeti na radu je da se usporede neki novi postojeći algoritmi te prilagođeniji algoritmi zadanoj domeni. Također, usporedba s glazbenim listama napravljenima koristeći umjetnu inteligenciju i neuronske mreže uvidjela bi na još neke mogućnosti. Kod usporedbe s ručno izrađenom glazbenom listom trebalo bi se usporediti s mnogo više različitih ručno izabranih glazbenih lista da se može stvarno pokazati je li uvijek uspješnije.

Zaključak

U ovom radu istražena je mogućnost izrade glazbene liste primjenjujući algoritme višekriterijske optimizacije. Problem generiranja liste kategoriziran je kao višekriterijski diskretni optimizacijski problem s tri kriterija: broj različitih izvođača, prosječnom popularnošću pjesama te koherentnošću audio značajki. Implementirana su tri evolucijska algoritma: NSGA-II, NSGA-III te MOEA/D. Njihova učinkovitost je izmjerena koristeći metrike konvergencije i disperzije rješenja: generacijsku udaljenost, „spacing“ te hipervolumen. Metrike su mjerene nad trideset neovisnih pokretanja svakog algoritma.

Rezultati usporedbe algoritama pokazuju da su sva tri algoritma uspješno savladala zadani problem. NSGA-III algoritam proizveo je najbolje rezultate s vrijednošću hipervolumena 0.4429 ± 0.1165 . Algoritam MOEA/D ostvario je bolju mjeru razmaka, no uz znatno veću disperziju rezultata između različitih pokretanja. Algoritam NSGA-II postiže vrlo slične rezultate kao NSGA-III, no malo slabije po svim mjerama. Usporedba s nasumično generiranim glazbenim listama pokazuje uspješnost algoritama, gdje je konvergencija bila znatno bolja. Mjera razmaka je slična u oba slučaja što je očekivani rezultat budući da nasumična rješenja pokrivaju prostor bez usmjeravanja prema optimalnim rješenjima. Usporedba s ručno izrađenom glazbenom listom pokazuje da NSGA-III algoritam u našem slučaju uspješniji. To pokazuje da algoritamsko biranje glazbenih lista moglo biti bolje od ručnog biranja glazbenih lista.

Zaključno, rad pokazuje da je primjena evolucijskih algoritama učinkovit pristup generiranju glazbenih lista. Najprigodniji algoritam pokazao se NSGA-III, no sva tri algoritma su uspješno pronalazila kompromisna rješenja. Uz moguća poboljšanja, kao što su usporedba s još nekim postojećim algoritmima, implementacija novog algoritma u ovoj domeni, te usporedba sa strojnim učenjem, ovaj rad postavlja solidnu osnovu za daljnje istraživanje problema automatskog generiranja glazbenih lista.

Sažetak

Višekriterijska optimizacija glazbenih lista pomoću evolucijskih algoritama

U ovom radu implementirani su evolucijski algoritmi za višekriterijsku optimizaciju glazbenih lista – NSGA-II, NSGA-III, MOEA/D - u programskom jeziku Python. Algoritmi su međusobno uspoređeni primjenom mjera raznolikosti i konvergencije – hipervolumenom, generacijskom udaljenošću i mjerom razmaka. Korištene su da bi se procijenila njihova učinkovitost u optimizaciji glazbenih lista. Dobiveni rezultati prikazani su tablično i grafički korištenjem Matplotlib modula te pokazuju da primjena višekriterijske optimizacije uspješna u generiranju optimiziranih glazbenih lista. Analiza također omogućuje usporedbu performansi implementiranih algoritama s obzirom na kvalitetu dobivenih rješenja.

Ključne riječi: Višekriterijska optimizacija, Glazbene liste, Spotify, NSGA-II, NSGA-III, MOEA/D, Hipervolumen

Summary

Multiobjective optimization of playlists using evolutionary algorithms

This thesis presents the implementation of multi-objective optimization algorithms, namely NSGA-II, NSGA-III and MOEA/D, using the Python language. The algorithms are compared using convergence and diversity metrics which are generational distance, spacing and most importantly hypervolume. These were used to evaluate their effectiveness in playlist optimization. The obtained results are presented in both tabular and graphical form using the Matplotlib library and demonstrate that the proposed multi-objective optimization approach successfully generates optimized playlists. Furthermore, the analysis provides a comparison of the implemented algorithms based on the quality of the obtained solutions.

Keywords: Multi-objective optimization, playlists, Spotify, NSGA-II, NSGA-III, MOEA/D, hypervolume

Literatura

Baza podataka

:

[3] <https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs>

Tekst

[1] <https://en.wikipedia.org/wiki/Spotify>

[2] <https://hr.wikipedia.org/wiki/Playlista>

[4] https://en.wikipedia.org/wiki/Program_optimization

[5] <https://www.fer.unizg.hr/download/repository/01-optimizacija.pdf>

[6] https://hr.wikipedia.org/wiki/Evolucijski_algoritam

[7] <https://pymoo.org/algorithms/moo/nsga3.html>

[8] <https://medium.com/optuna/an-introduction-to-moea-d-and-examples-of-multi-objective-optimization-comparisons-8630565a4e89>

[9] <https://www.sciencedirect.com/science/article/pii/S2590123025025897>

[10] <https://pmc.ncbi.nlm.nih.gov/articles/PMC8514472/>

[11] <https://colab.ws/articles/10.1145/3453474>

Slike

(Sl. 1.1 Spotify glazbena lista) <https://open.spotify.com/>

(Sl. 2.1 Pareto fronta) https://en.wikipedia.org/wiki/Pareto_front

(Sl. 2.2 „Crowding distance“ NSGA-II i NSGA-III algoritama)

<https://www.fer.unizg.hr/download/repository/10%20Vi%C5%A1ekriterijska%20optimizacija.pdf>

(Sl. 3.1 Generacijska udaljenost) <https://pmc.ncbi.nlm.nih.gov/articles/PMC8514472/>

(Sl. 3.2 Hipervolumen) https://www.researchgate.net/figure/Hypervolume-indicator_fig3_334315629